



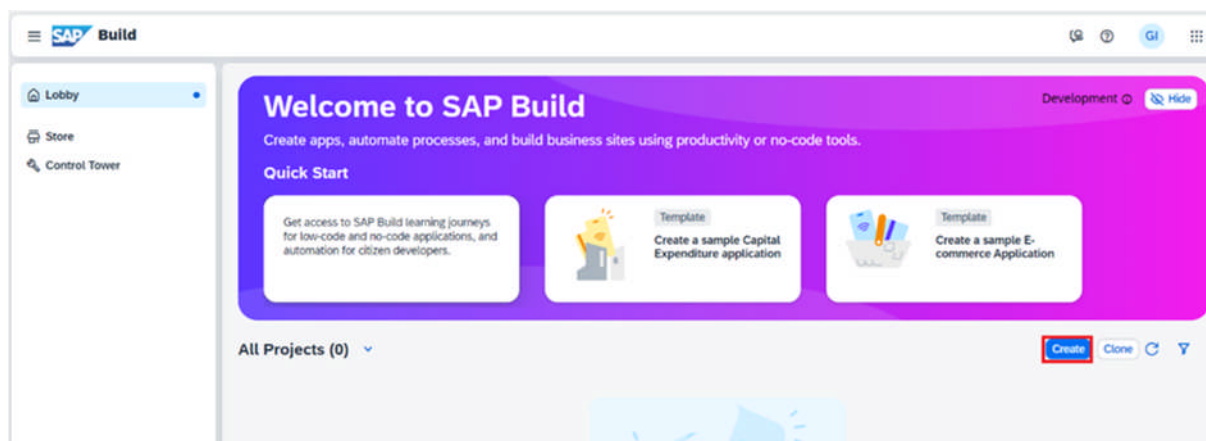
LAB MANUAL 2

Create a Full-Stack SAP Fiori Application with Joule in SAP Build Code

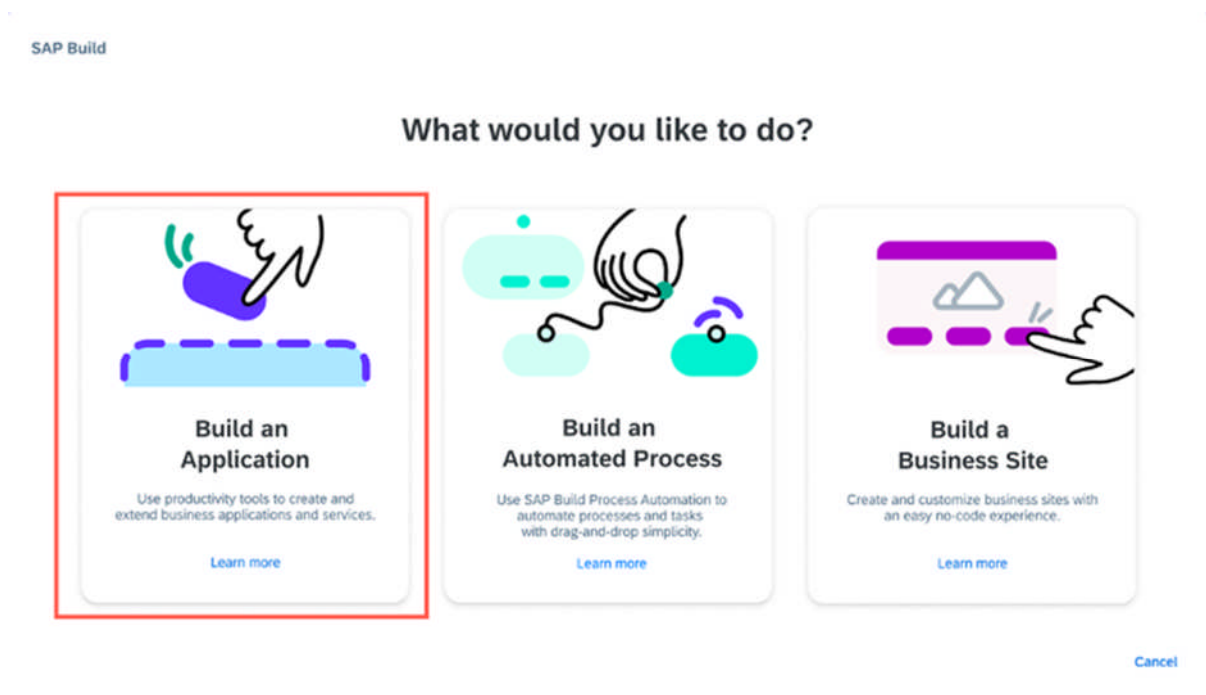
Lab:-2 Create a Full-Stack SAP Fiori Application with Joule in SAP Build Code

1 Create a New Project Using SAP Build Code

1. Navigate to the SAP Build lobby.
2. Click **Create** to start the creation process

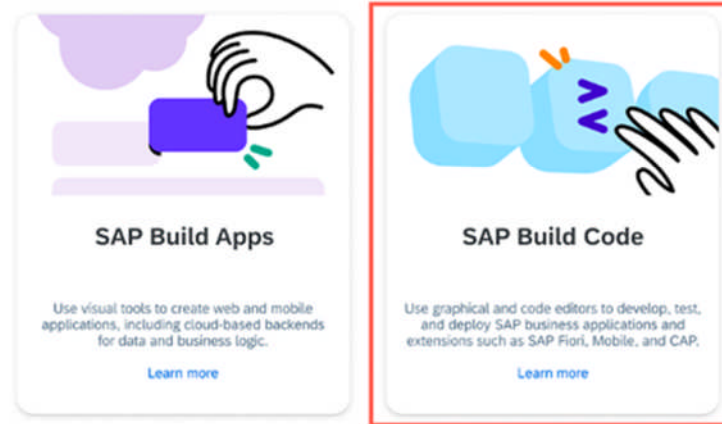


3. Click the **Build an Application** tile.



4. Click the **SAP Build Code** tile to develop your project in SAP Business Application Studio, the SAP Build Code development environment, leveraging the capabilities of the services included in SAP Build Code.

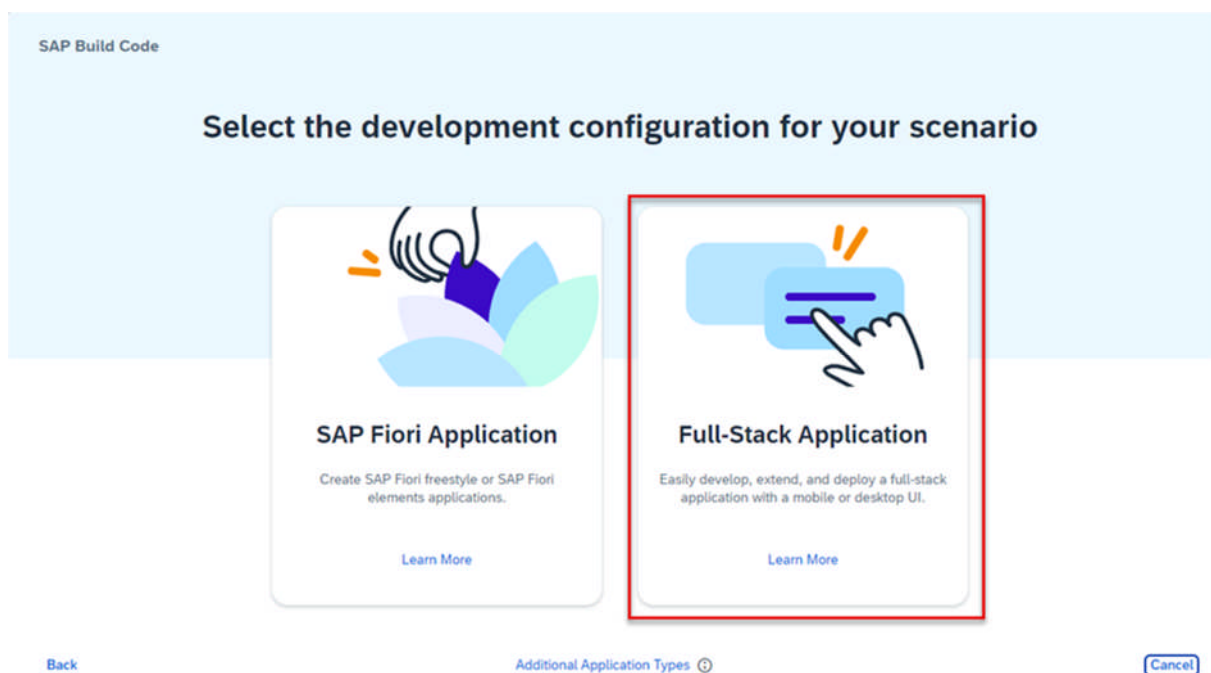
How do you want to build your app?



[Back](#)

[Cancel](#)

5. Click the **Full-Stack Application** tile.



6. Enter a name for your project.
7. Select the dev space where you want the project to reside.
8. Click Create.

Create a **Full-Stack** project

Give your project a name

Project Name: *

My_New_Project

Description:

Select Dev Space: ⓘ

You can run up to two dev spaces at a time. For more options, go to the [Dev Space Manager](#)

[Back](#)

[Create](#)

[Cancel](#)

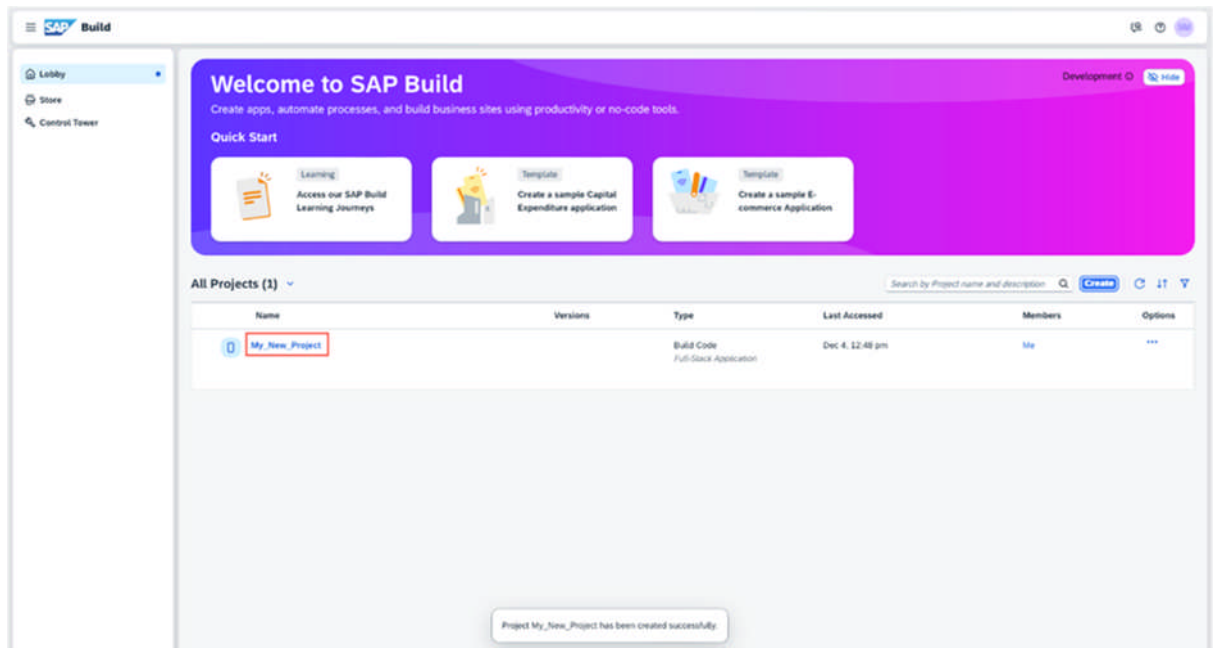
9. You can see the project being created in the Project table of the lobby.

The creation of the project may take a few moments.

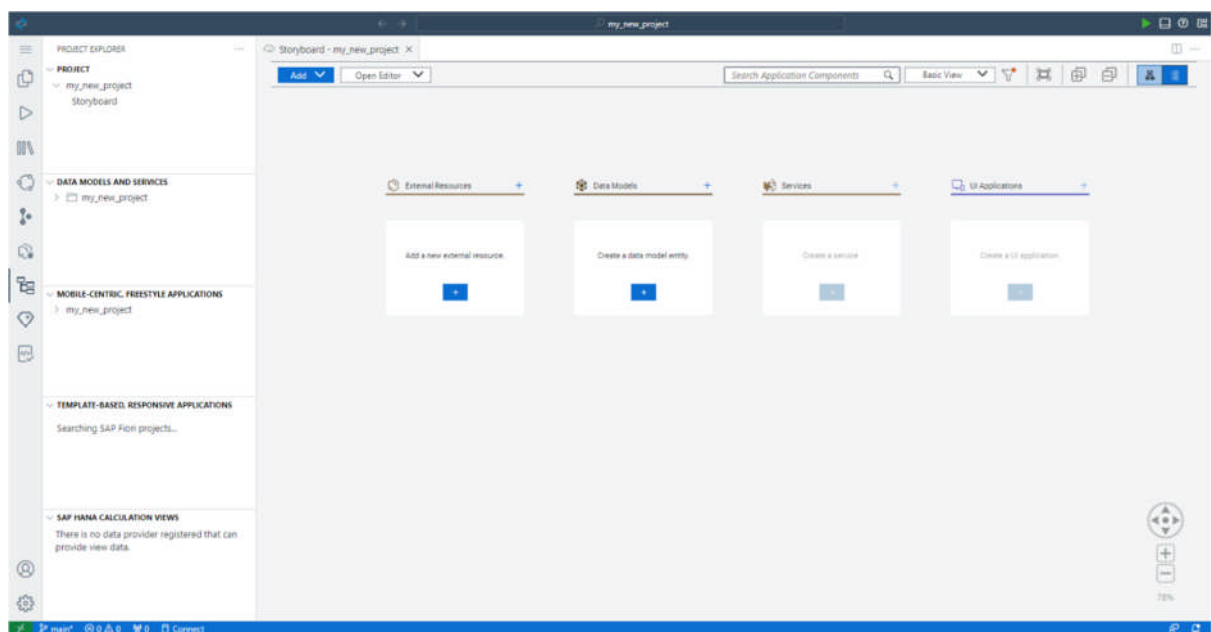
The screenshot shows the SAP Build interface. On the left is a sidebar with 'Lobby', 'Store', and 'Control Tower'. The main area has a 'Welcome to SAP Build' header with a 'Development' toggle and a 'Quick Start' section with three cards: 'Learning' (Access our SAP Build Learning Journeys), 'Template' (Create a sample Capital Expenditure application), and 'Template' (Create a sample E-commerce Application). Below this is a table titled 'All Projects (1)' with columns: Name, Versions, Type, Last Accessed, Members, and Options. The table contains one row for 'My_New_Project' with type 'Build Code Full-Stack Application', last accessed 'Dec 4, 12:48 pm', and member 'Me'. The 'Options' column shows a 'Creating...' status.

Name	Versions	Type	Last Accessed	Members	Options
My_New_Project		Build Code Full-Stack Application	Dec 4, 12:48 pm	Me	Creating...

10. After you see a message stating that the project has been created successfully, click the project to open it.



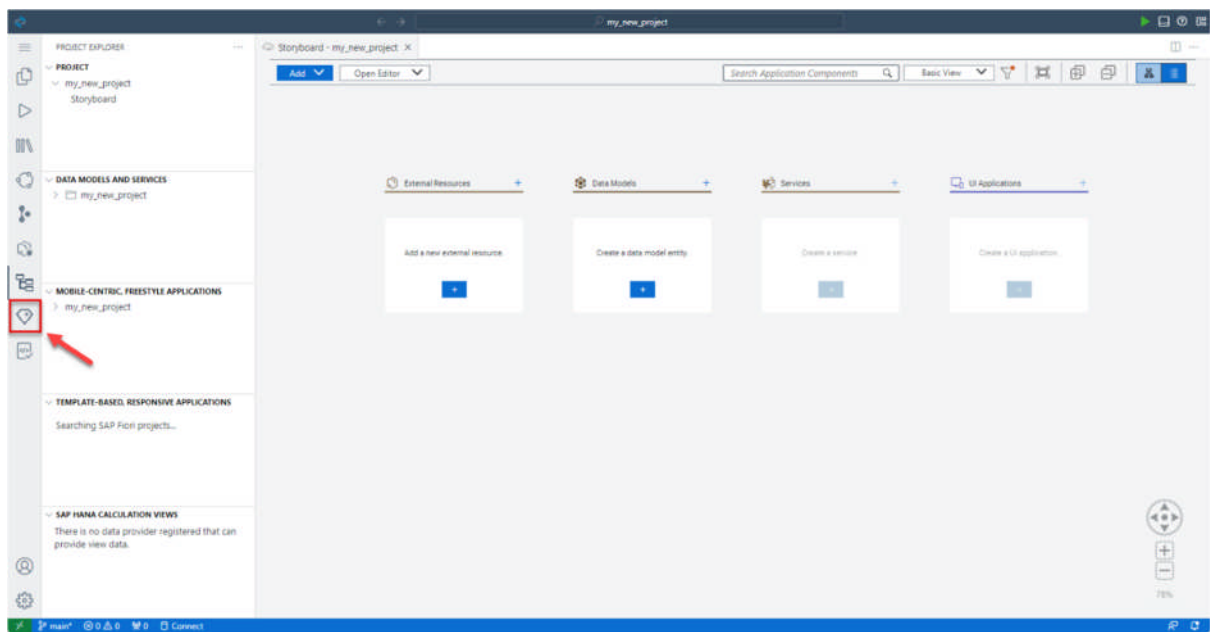
The project opens in SAP Business Application Studio, the SAP Build Code development environment.



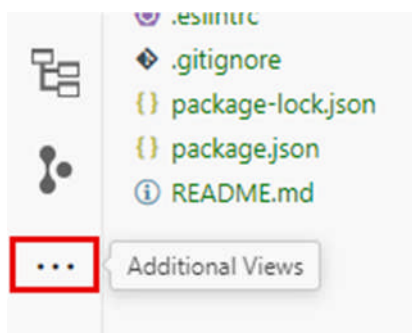
2 Create Data Entities with Joule

In SAP Business Application Studio, the SAP Build Code development environment, open the digital assistant, Joule, from the activity bar.

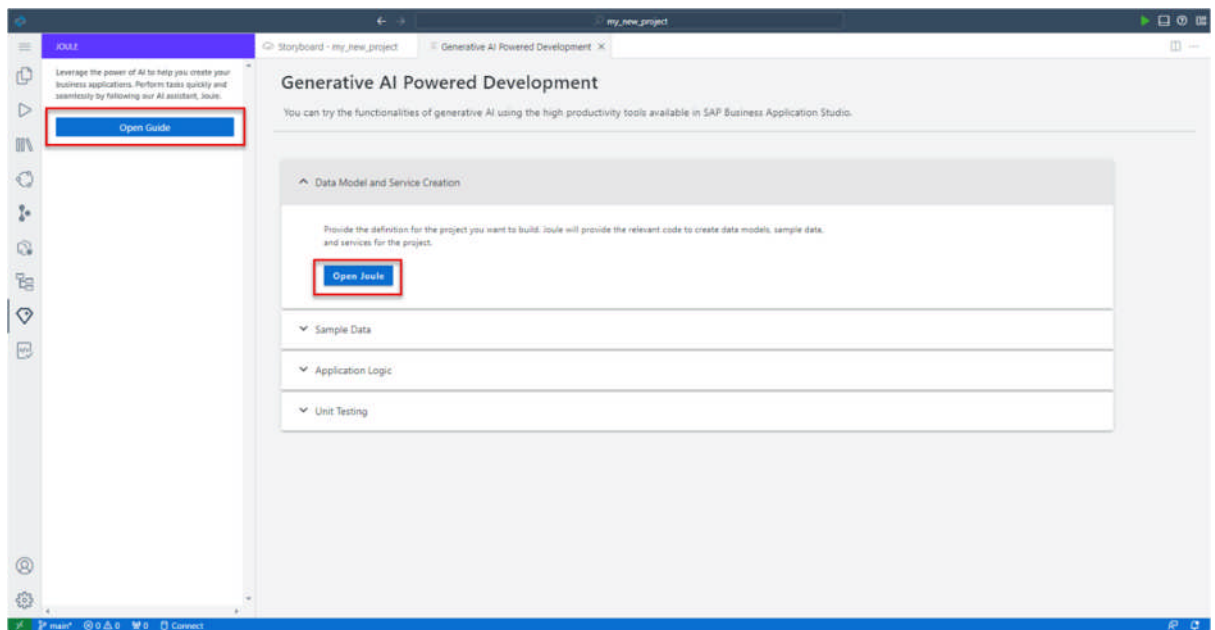
1. In SAP Business Application Studio, the SAP Build Code development environment, open the digital assistant, Joule, from the activity bar.



Note: If you do not see the icon, click Additional Views and select **Joule** from the list.



2. Click **Open Guide**.
3. Expand the **Data Model and Service Creation** section, and click **Open Joule**.



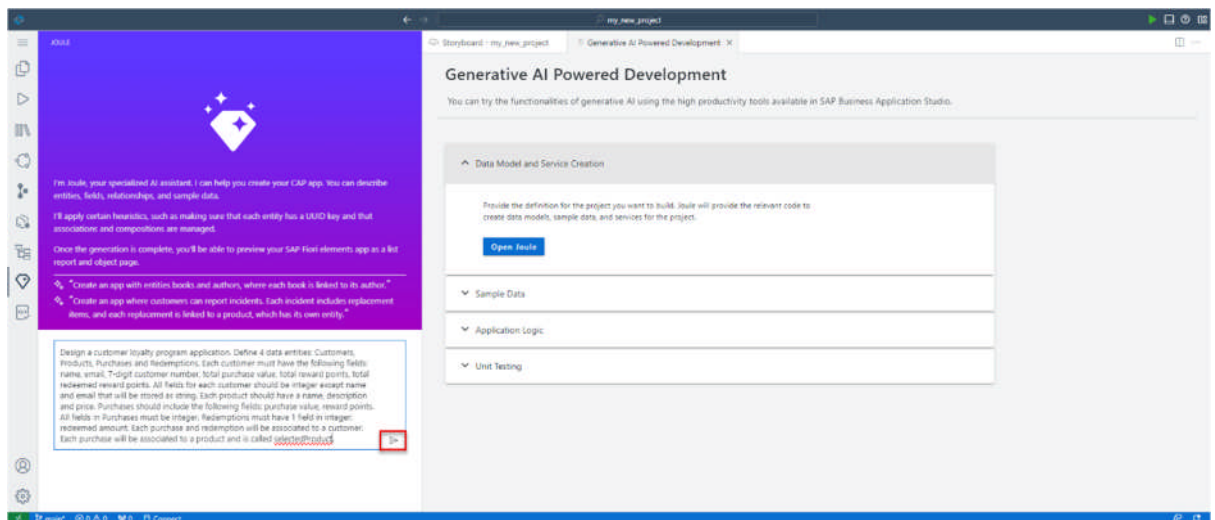
Copy the prompt below.

```
Design a customer loyalty program application.  
Define 4 data entities: Customers, Products, Purchases and Redemptions.  
Each customer must have the following fields: name, email, 7-digit  
customer number, total purchase value, total reward points, total  
redeemed reward points.  
All fields for each customer should be integer except name and email  
that will be stored as string.  
Each product should have a name, description and price.  
Purchases should include the following fields: purchase value, reward  
points.  
All fields in Purchases must be integer.  
Redemptions must have 1 field in integer: redeemed amount.
```

4. Paste the code in the text field, and click the arrow

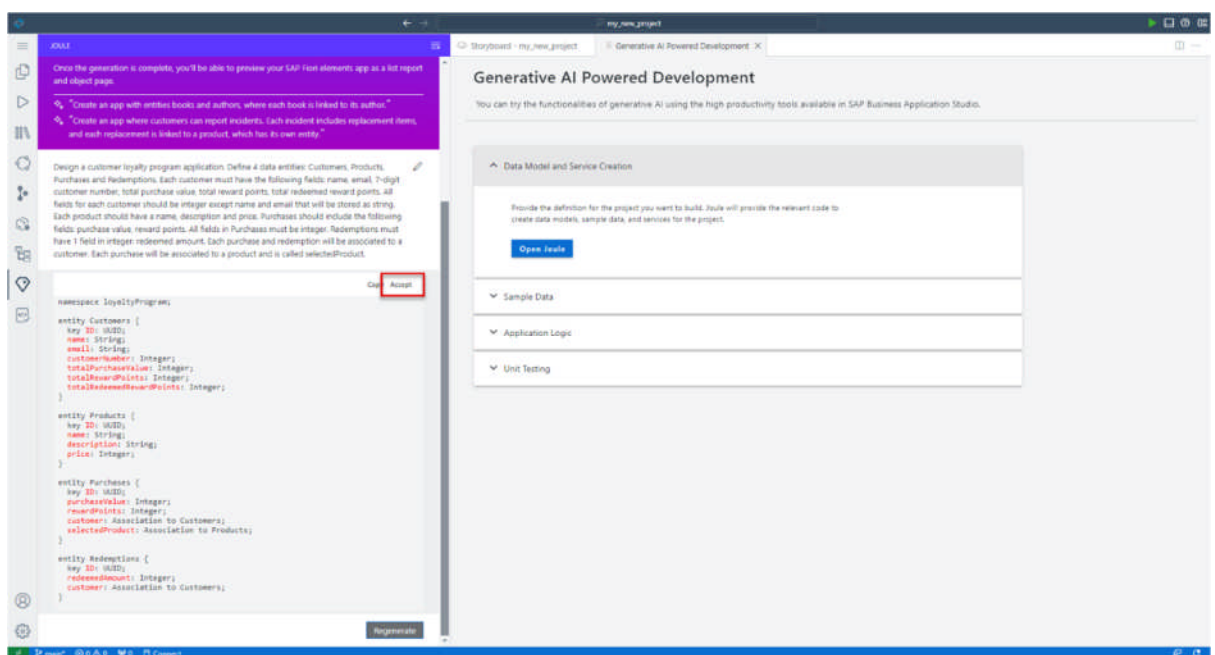


to send the prompt to Joule.



The code is generated and is displayed below your prompt.

6. Accept the code.



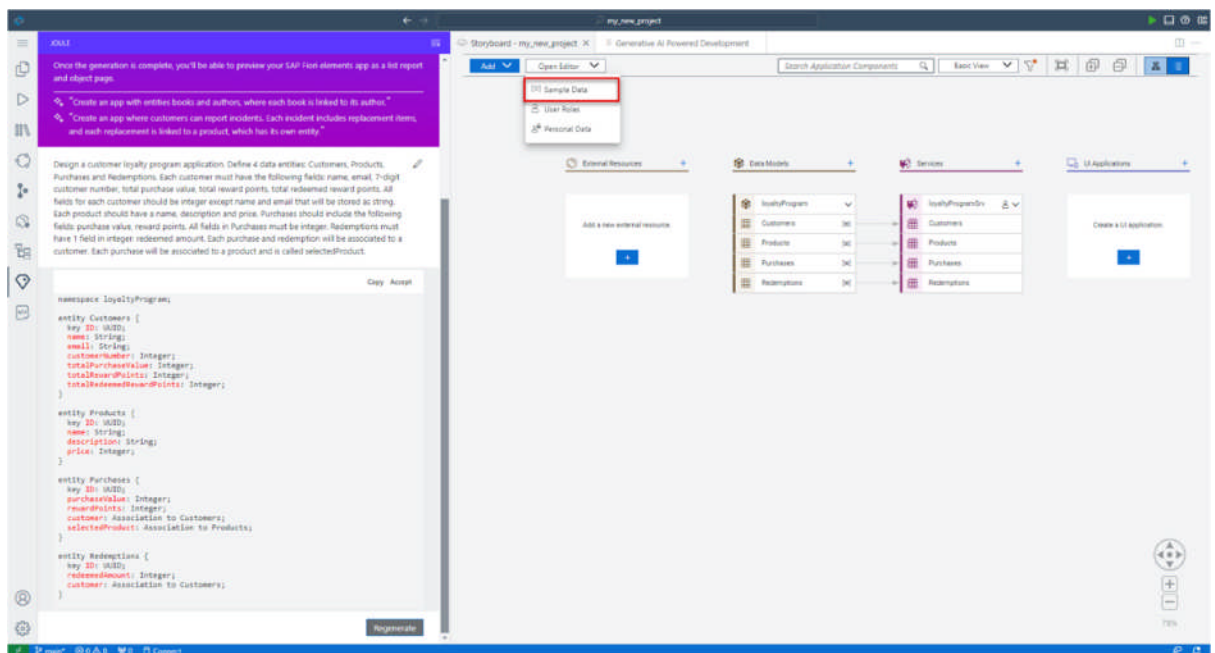
Depending on the server, it may take a few moments for Joule to create the data models and services for you.

Once you accept the code, you will see the update on the right side in the Storyboard tab.

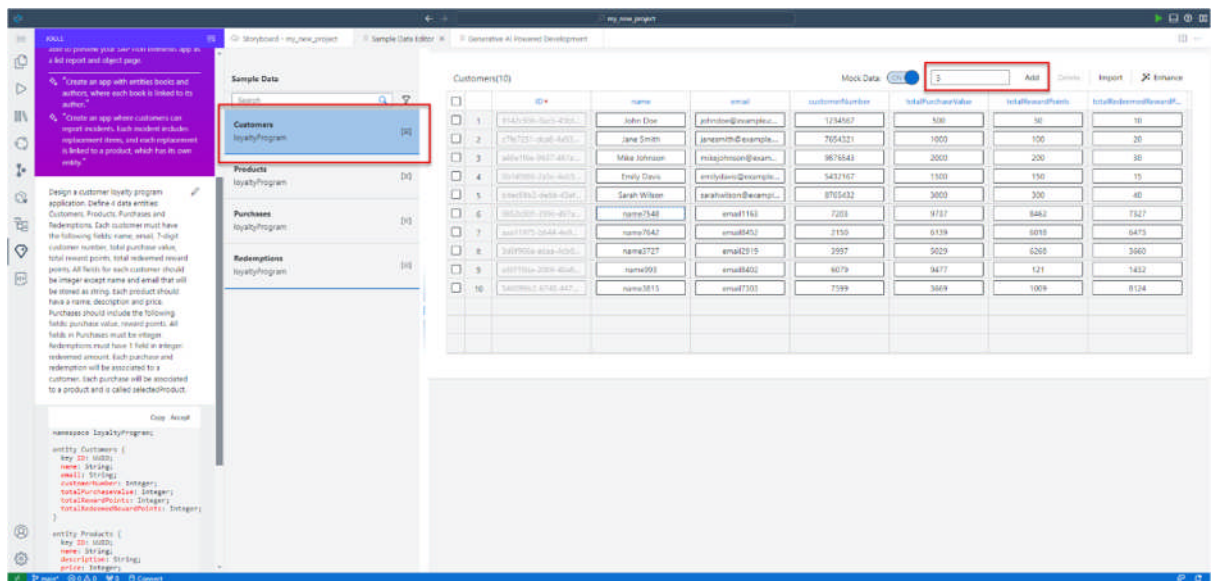
3 Enhance the Sample Data Using Joule

Joule created the CAP data model and the OData service. In addition, Joule created some sample data by default. We will now ask Joule to provide additional sample data.

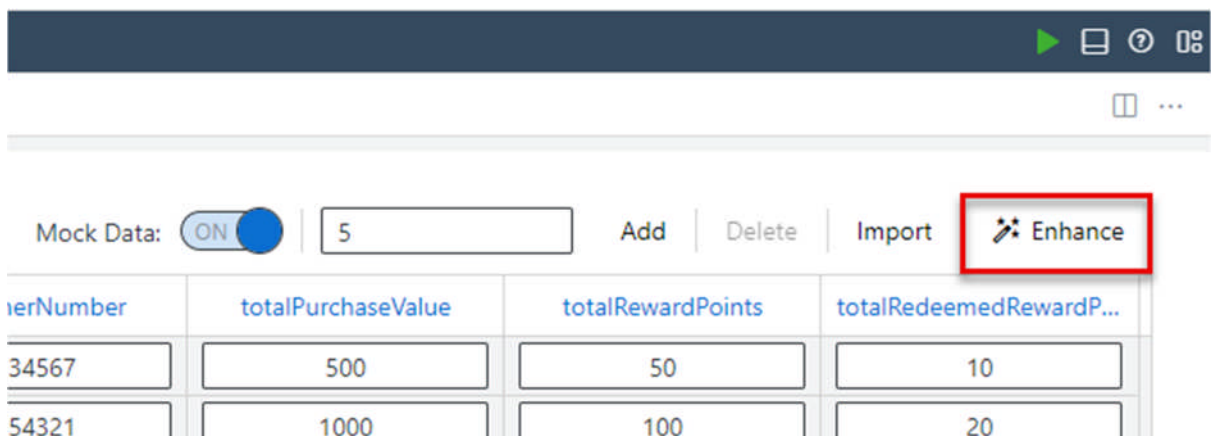
1. Open the Sample Data editor in the Storyboard by selecting **Open Editor -> Sample Data**.



2. In the Sample Data Editor, select the **Customers** data entity, and add 5 more rows. Click **Add**.



3. Click **Enhance**. This will reopen Joule to modify the sample data.



Copy the prompt below:

Enhance my sample data with meaningful data. Any phone numbers must be 10 digits long.
All customer numbers must be 7 digits long and one customer must use the customer number 1200547.
No fields may be empty.
Total purchase value must be smaller than 10000 not rounded.
Both total reward points and total redeemed reward points must not be rounded, must not be identical. and must always sum to one-tenth of the total purchase value for each customer.

Paste the prompt in the text field, and click the arrow (



) to send the prompt to Joule.

The screenshot shows the Joule AI assistant interface within a storyboard editor. The main panel has a purple background with the Joule logo (a white diamond with three stars) and introductory text. Below the text is a text input field containing a detailed prompt about enhancing sample data. A red box highlights a right-pointing triangle icon at the bottom right of the input field, which is used to send the prompt. To the right of the main panel is a sidebar titled 'Storyboard - my_new_project' with a 'Sample Data' section. This section contains a search bar and a list of data entities: Customers, Products, Purchases, and Redemptions, each with a 'loyaltyProgram' value and a collapse icon.

JOULE

I'm Joule, your specialized AI assistant. I can help you enhance sample data.

So, whenever you need help with enhancing sample data, just ask, and I'll be here to make your experience even better.

Caution: Adding too much sample data can lead to exceeding your AI quota and can cause performance issues.

Though I strive for perfection, I might not get everything right all the time.

❖ "Change 'books' entity to non-fiction relevant values."

❖ "Make sure the 'criticality' value in the range of [1(Negative), 2(Critical), 3(Positive)]."

Enhance my sample data with meaningful data. Any phone numbers must be 10 digits long. All customer numbers must be 7 digits long and one customer must use the customer number 1200547. No fields may be empty. Total purchase value must be smaller than 10000 not rounded. Both total reward points and total redeemed reward points must not be rounded, must not be identical, and must always sum to one-tenth of the total purchase value for each customer.

Storyboard - my_new_project Sample Data

Search

Customers loyaltyProgram

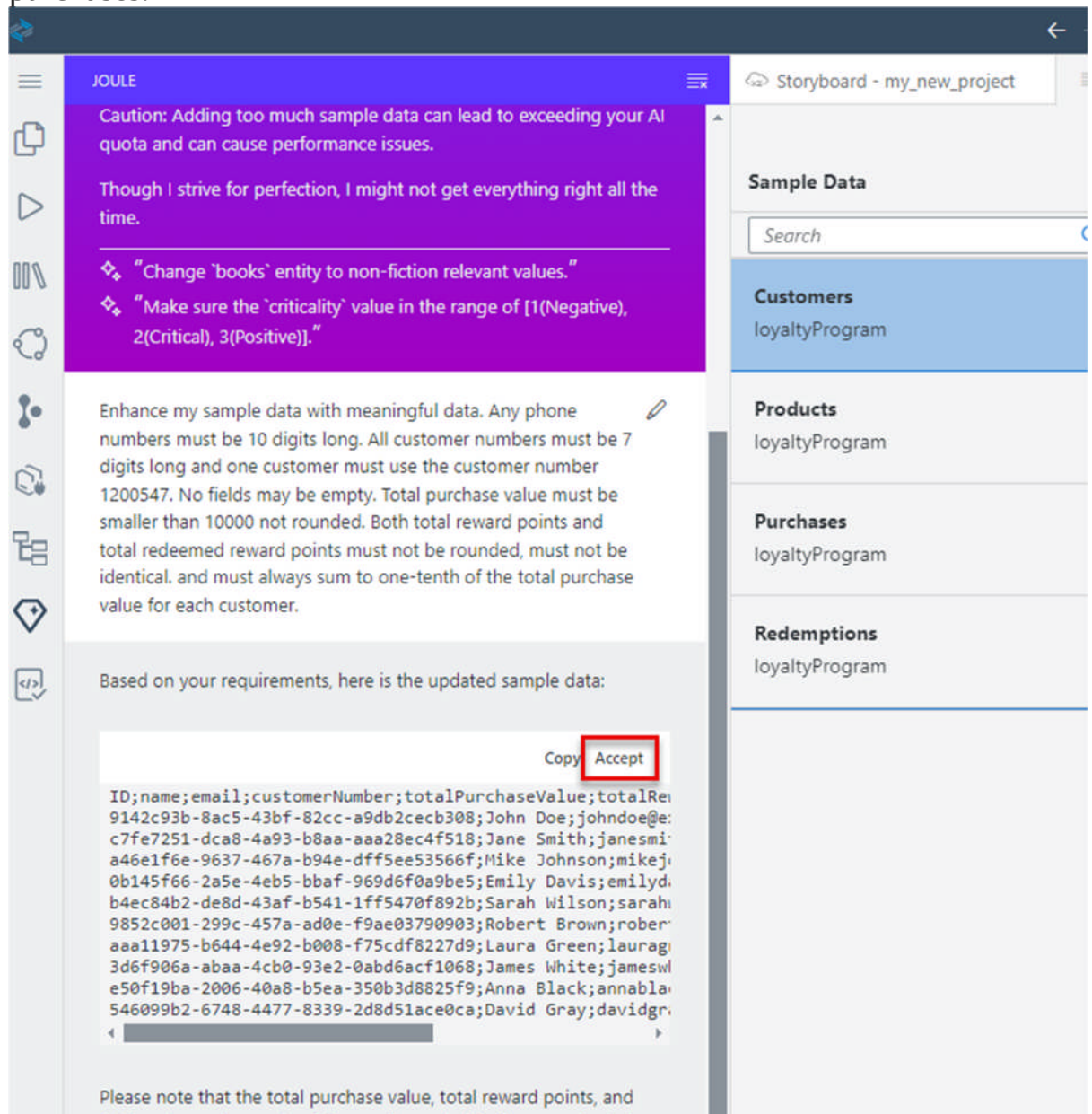
Products loyaltyProgram

Purchases loyaltyProgram

Redemptions loyaltyProgram

The code is generated and is displayed below your prompt.

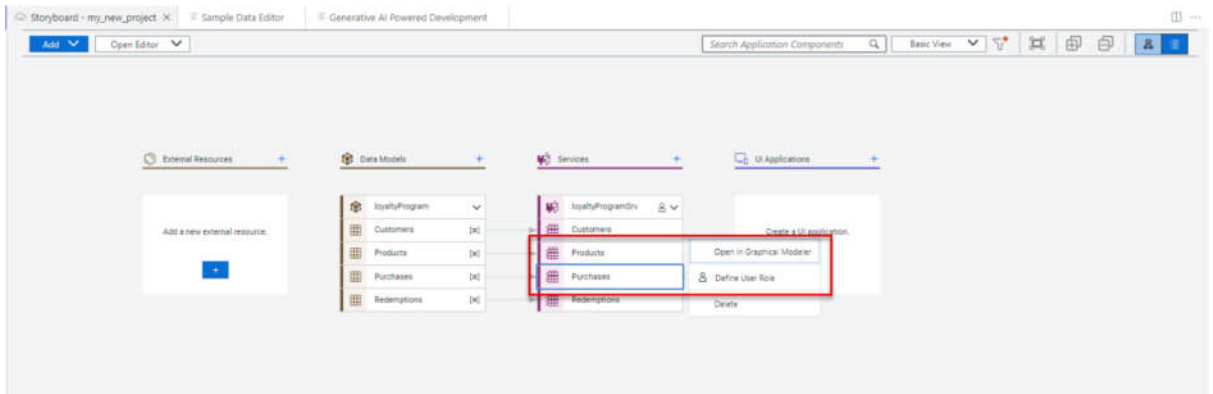
4. Accept the code. This will add the customer names, email addresses, and purchases.



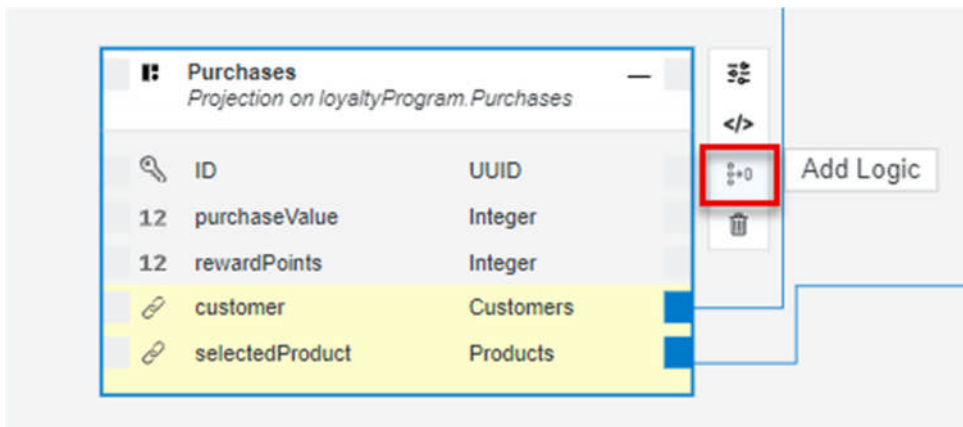
4 Create Application Logic with Joule

We already have created the data model, service, and sample data with Joule. Now we want to create some logic for our service. We would like to calculate the bonus points automatically when a customer makes a purchase. Additionally, we want to provide logic for customers to redeem these bonus points.

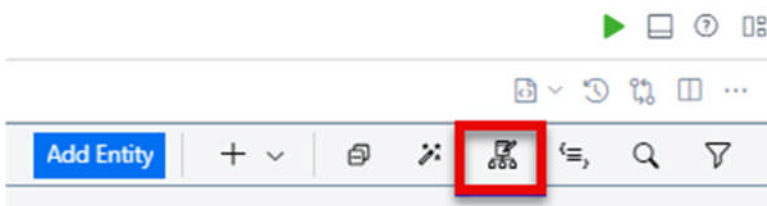
1. In the Storyboard, click on the **Purchases** entity under **Services**, and select **Open in Graphical Modeler**.



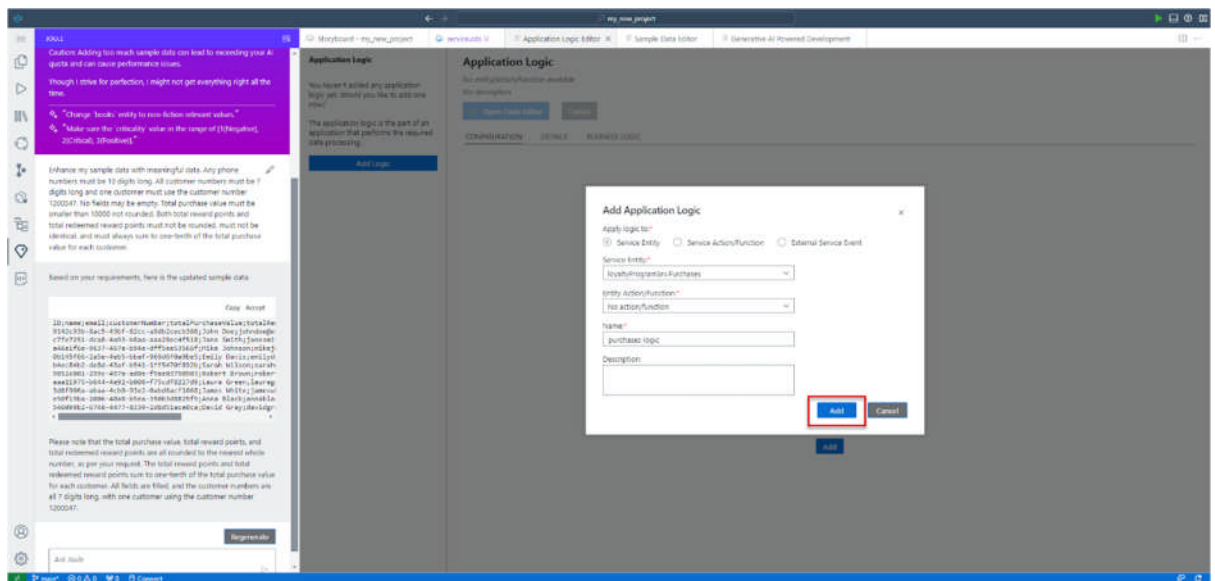
2. Select the **Purchases** entity by clicking on the title. Then, click **Add Logic**.



If you do not see the entity, click the Show All icon.

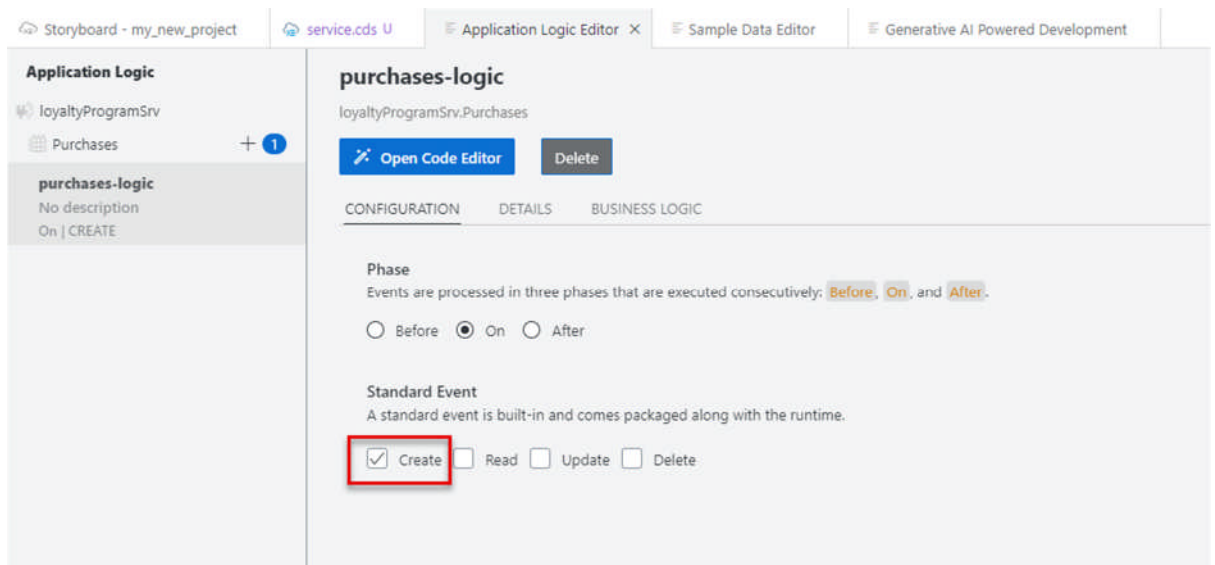


3. In the **Add Application Logic** dialog, leave the default values, and click **Add**.

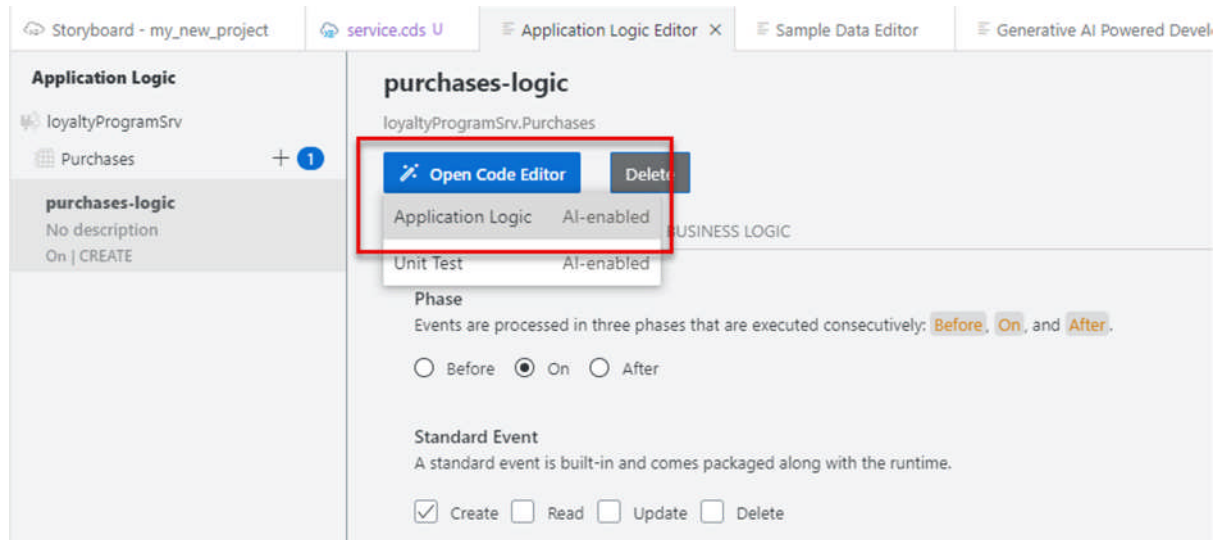


The Application Logic Editor opens.

4. In the **Standard Event** section, select **Create**. That means that this logic will be automatically executed if an OData create operation is requested.



5. Click **Open Code Editor**, and select **Application Logic**. This will open Joule again to allow us to send a prompt to Joule to create the logic for us.



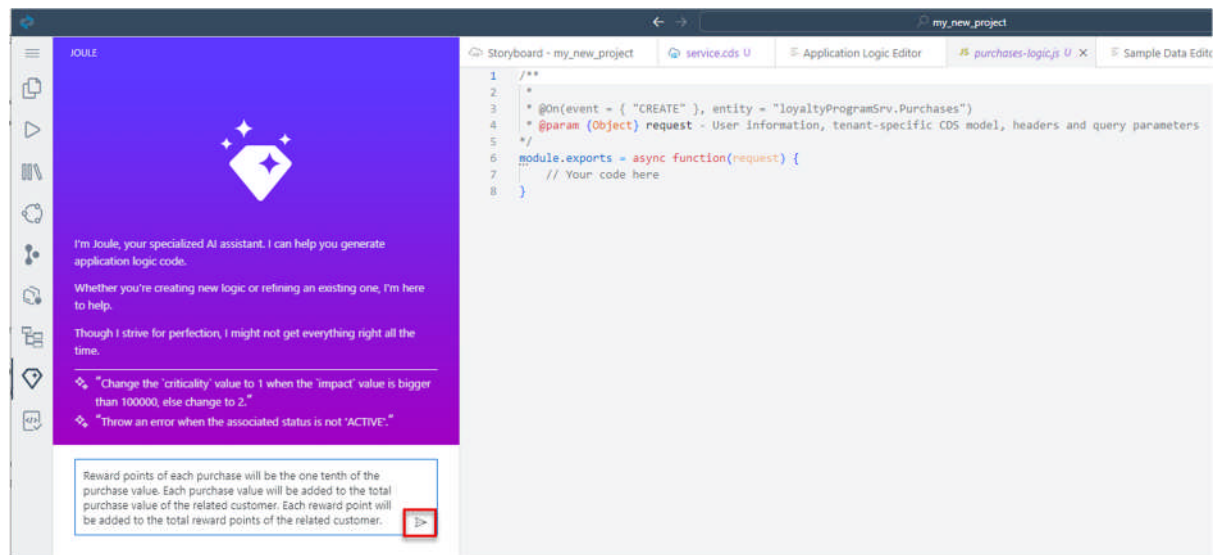
Copy the prompt below:

Reward points of each purchase will be the one tenth of the purchase value.
Each purchase value will be added to the total purchase value of the related customer.
Each reward point will be added to the total reward points of the related customer.

Paste the prompt in the text field, and click the arrow (



) to send the prompt to Joule.



So Joule created code that implements the following logic:

- o Check if the customer exists
- o Calculate the rewardPoints from the purchase value
- o Update the total purchase value and the total reward points in the customers entity

6. Accept the code.

Whether you're creating new logic or refining an existing one, I'm here to help.

Though I strive for perfection, I might not get everything right all the time.

- ❖ "Change the `criticality` value to 1 when the `impact` value is bigger than 100000, else change to 2."
- ❖ "Throw an error when the associated status is not 'ACTIVE'."

Reward points of each purchase will be the one tenth of the purchase value. Each purchase value will be added to the total purchase value of the related customer. Each reward point will be added to the total reward points of the related customer.

Here is the custom logic function that meets your requirements:

Copy **Accept**

```
/**
 *
 * @On(event = { "CREATE" }, entity = "loyaltyProgramSr
 * @param {Object} request - User information, tenant-s
 */
module.exports = async function(request) {
  // Calculate reward points based on purchase value
  request.data.rewardPoints = Math.floor(request.data

  // Update the total purchase value and total reward
  const customer = await SELECT.one('loyaltyProgramSr
    .where({ ID: request.data.customer_ID });

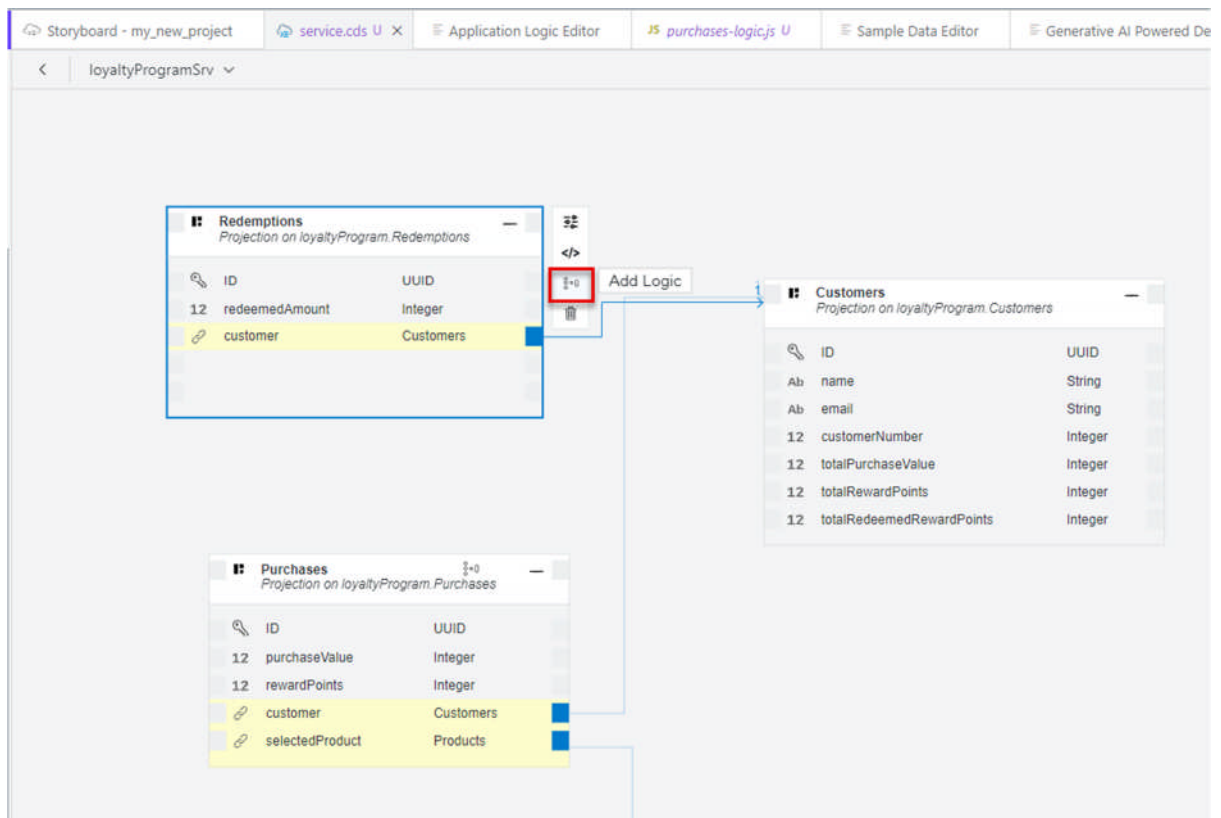
  if (customer) {
    const updatedCustomer = {
      ID: customer.ID,
      totalPurchaseValue: customer.totalPurchaseV
      totalRewardPoints: customer.totalRewardPoin
    };

    await UPDATE('loyaltyProgramSrv.Customers')
      .set(updatedCustomer)
      .where({ ID: updatedCustomer.ID });
  }
}
```

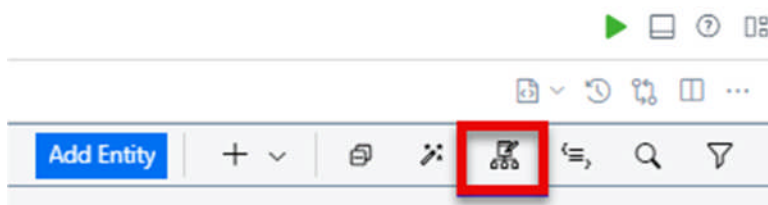
This function first calculates the reward points based on the purchase value. Then it retrieves the related customer and updates their total purchase value and total reward points.

Note: Joule typically generates different code each time for the same prompt. If yours is different to what you can see here, that's fine as long as it does the same job. If there are no obvious errors, just keep working on the exercise. If you aren't sure, you can ask Joule to try again by clicking **Regenerate**.

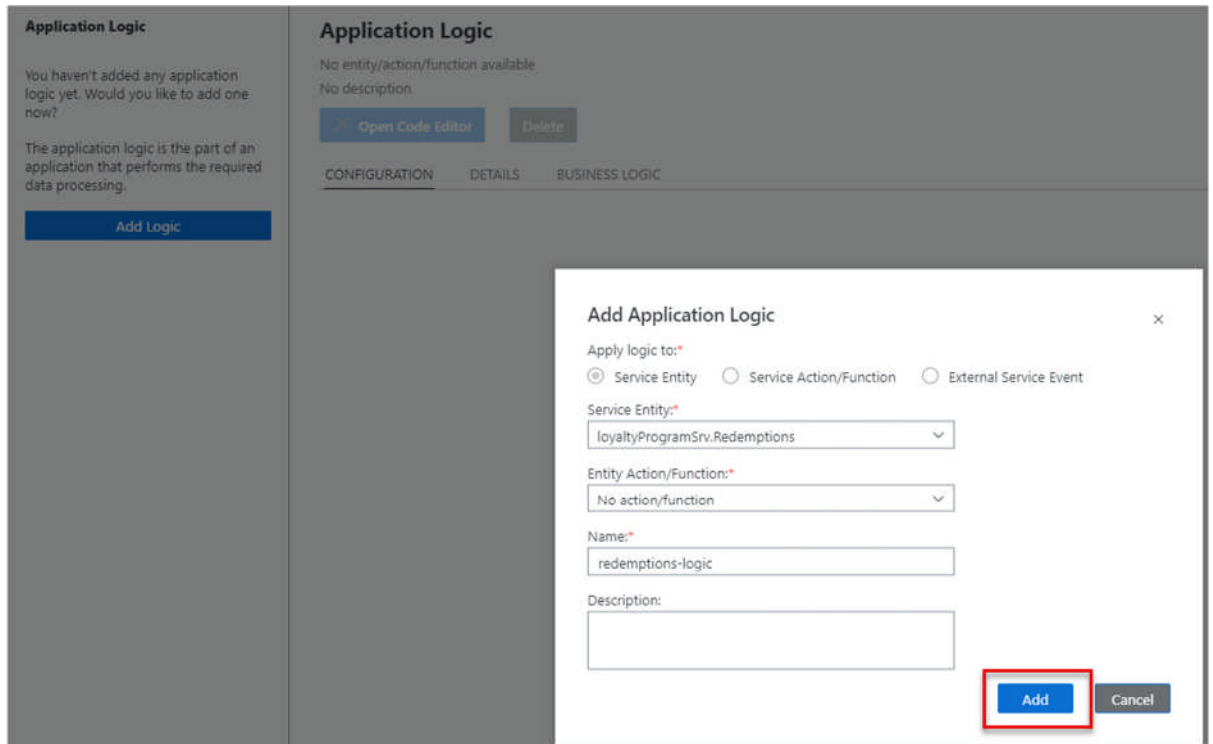
7. Go back to the **service.cds** tab.
8. Select the **Redemptions** entity by clicking on the title. Then, click **Add Logic**.



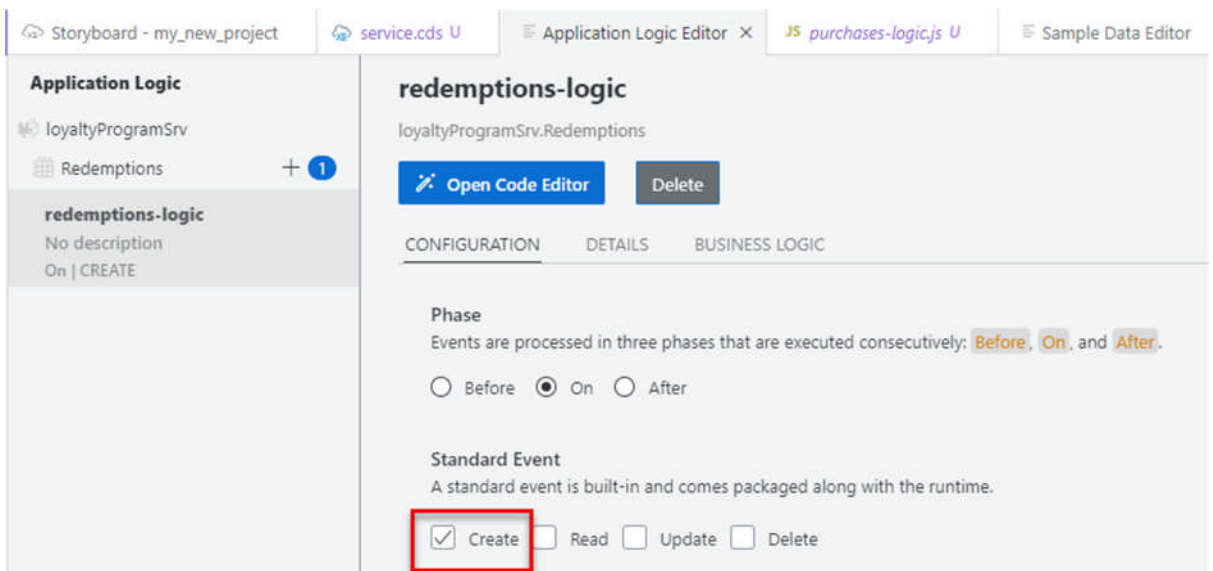
If you do not see the entity, click the Show All icon.



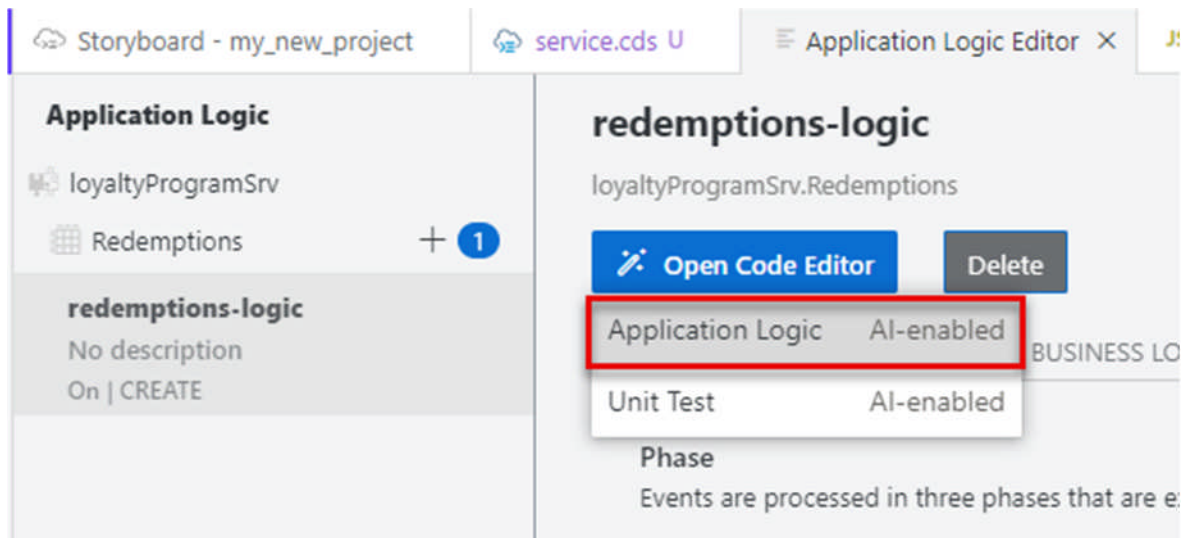
9. In the **Add Application Logic** dialog, leave the default values, and click **Add**.



10. In the **Standard Event** section, select **Create**.



11. Click **Open Code Editor**, and select **Application Logic**. This will open Joule again to allow us to send a prompt to Joule to create the logic for us.



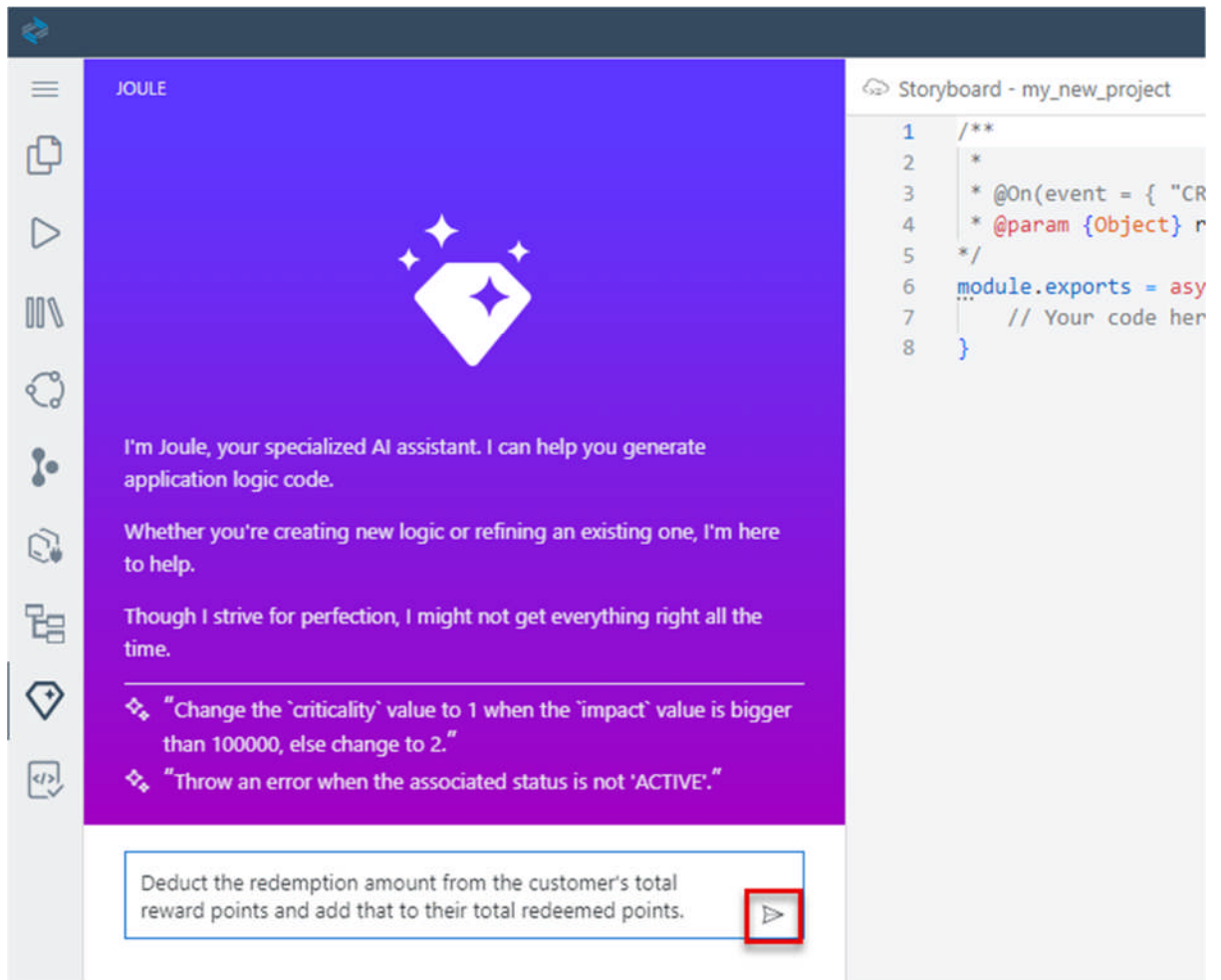
Copy the prompt below::

Deduct the redemption amount from the customer's total reward points and add that to their total redeemed points.

Paste the prompt in the text field, and click the arrow (



) to send the prompt to Joule.



16.

The screenshot shows the JOULE IDE interface. On the left, a sidebar contains various icons for file management, execution, and settings. The main editor area is divided into two panes. The top pane, titled 'Storyboard - my_new_project', displays a visual storyboard with a purple background. It contains a text box with the sentence 'Though I strive for perfection, I might not get everything right all the time.' Below this, there are two diamond-shaped logic blocks. The first block contains the text: 'Change the 'criticality' value to 1 when the 'impact' value is bigger than 100000, else change to 2.' The second block contains the text: 'Throw an error when the associated status is not 'ACTIVE'.' Below these blocks, there is a text box with the instruction: 'Deduct the redemption amount from the customer's total reward points and add that to their total redeemed points.' Below this, there is a text box with the instruction: 'Here is the custom logic function that you can use:'. To the right of this text box, there are two buttons: 'Copy' and 'Accept'. The 'Accept' button is highlighted with a red rectangle. Below the text box, there is a code editor with a light blue background. It contains a custom logic function written in JavaScript. The function is an asynchronous function that takes a request object as an argument. It first extracts the redemption amount and customer ID from the request data. Then, it uses the CDS transaction to fetch the customer's current reward points and redeemed reward points. It then checks if the customer has enough reward points to redeem. If not, it throws an error. If yes, it deducts the redemption amount from the customer's total reward points and adds it to their total redeemed points. The function is then exported as an asynchronous function.

```

1  /**
2   *
3   * @On(event = { "CREATE" }, entity = "loyaltyProgramSrv"
4   * @param {Object} request
5   */
6  module.exports = async function(req) {
7    // Your code here
8  }

```

```

/**
 *
 * @On(event = { "CREATE" }, entity = "loyaltyProgramSrv"
 * @param {Object} req - User information, tenant-speci
 */
module.exports = async function(req) {
  const { redeemedAmount, customer_ID } = req.data;
  const tx = cds.transaction(req);

  // Fetch the customer's current reward points and r
  const [{ totalRewardPoints, totalRedeemedRewardPoint
    SELECT.from('loyaltyProgramSrv.Customers')
      .where({ ID: customer_ID })
      .columns('totalRewardPoints', 'totalRedeeme
  });

  // Check if the customer has enough reward points to
  if (totalRewardPoints < redeemedAmount) {
    req.error(400, 'Insufficient reward points');
    return;
  }

  // Deduct the redeemed amount from the customer's t
  // and add it to their total redeemed points
  await tx.run(
    UPDATE('loyaltyProgramSrv.Customers')
      .set({
        totalRewardPoints: totalRewardPoints - r
        totalRedeemedRewardPoints: totalRedeeme
      })
      .where({ ID: customer_ID })
  );
};

```

This function will be triggered when a new redemption is created. It first fetches the customer's current reward points and redeemed points. Then it checks if the customer has enough reward points to

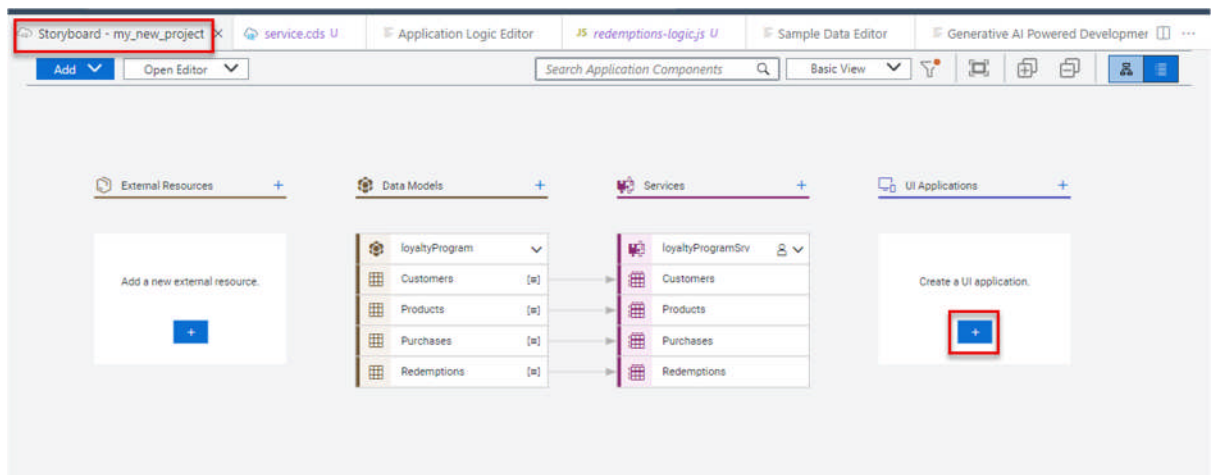
Have a closer look at the generated code. It even includes some checks to see if the customer has enough points to redeem.

```
1  /**
2   *
3   * @On(event = { "CREATE" }, entity = "loyaltyProgramSrv.Redemptions")
4   * @param {Object} req - User information, tenant-specific CDS model, headers and query parameters
5   */
6  module.exports = async function(req) {
7      const { redeemedAmount, customer_ID } = req.data;
8      const tx = cds.transaction(req);
9
10     // Fetch the customer's current reward points and redeemed points
11     const [{ totalRewardPoints, totalRedeemedRewardPoints }] = await tx.run(
12         SELECT.from('loyaltyProgramSrv.Customers')
13         .where({ ID: customer_ID })
14         .columns('totalRewardPoints', 'totalRedeemedRewardPoints')
15     );
16
17     // Check if the customer has enough reward points to redeem
18     if (totalRewardPoints < redeemedAmount) {
19         req.error(400, 'Insufficient reward points');
20         return;
21     }
22
23     // Deduct the redeemed amount from the customer's total reward points
24     // and add it to their total redeemed points
25     await tx.run(
26         UPDATE('loyaltyProgramSrv.Customers')
27         .set({
28             totalRewardPoints: totalRewardPoints - redeemedAmount,
29             totalRedeemedRewardPoints: totalRedeemedRewardPoints + redeemedAmount
30         })
31         .where({ ID: customer_ID })
32     );
33 }
```

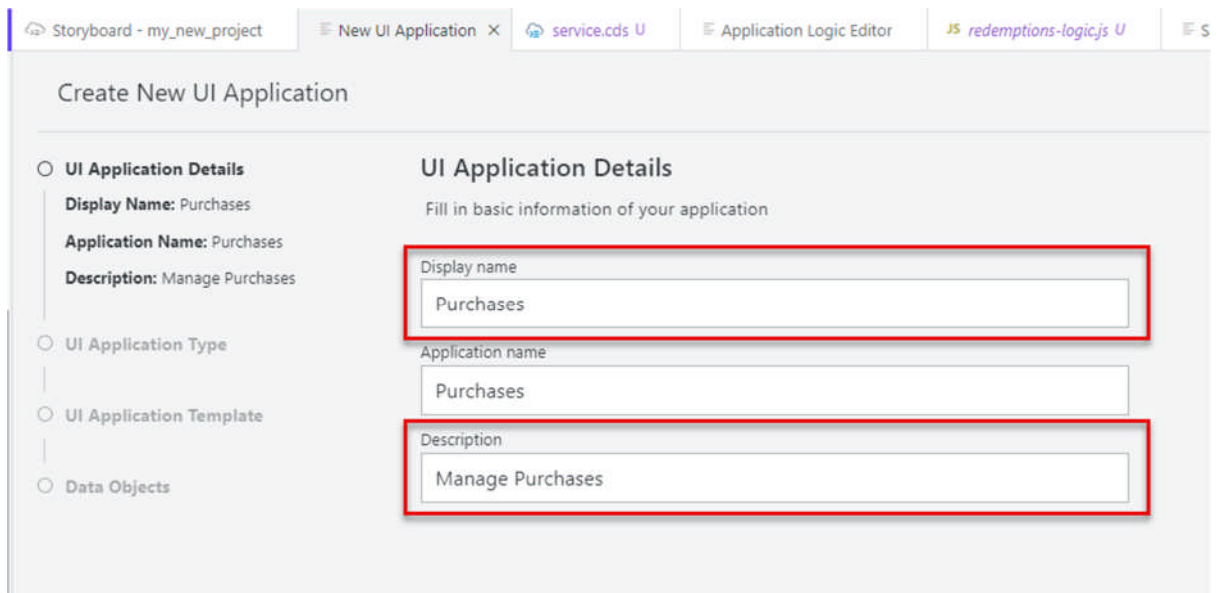
5 Add UI to the Application

To display and test the content we created for the customer loyalty program, we need to create an SAP Fiori elements UI.

1. Go to back to the Storyboard and add a UI application.



2. We will start with the user interface for the **Purchases** data entity. Set the **Display name** to **Purchases** and the **Description** to **Manage Purchases**, and then click **Next**.



3. We are using the browser, so we will select **Template-Based Responsive Application** as the UI Application type, and click **Next**.

Storyboard - my_new_projectNew UI Application ×service.cds UApplication Logic EditorJS redemptions-logic.js USampl

Create New UI Application

UI Application Details

Display Name: Purchases

Application Name: Purchases

Description: Manage Purchases

UI Application Type

What Kind Of Application Do You Want To Create?: Template-Based, Responsive Application

UI Application Template

Data Objects

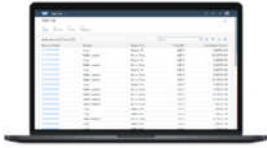
UI Application Type

What kind of application do you want to create? *

Template-Based, Responsive

Create a browser-based application with standard yet extensible floorplans, runs on desktop and mobile. It is derived from your data structures and metadata, automatically applies the latest SAP Fiori design.

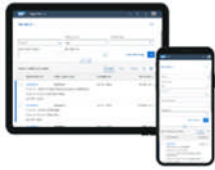
[More information](#)



Mobile-Centric, Freestanding

Create an application to run natively on mobile (Android & iOS) with device specific features such as biometric authentication and barcode scanning. It also runs on desktop browsers.

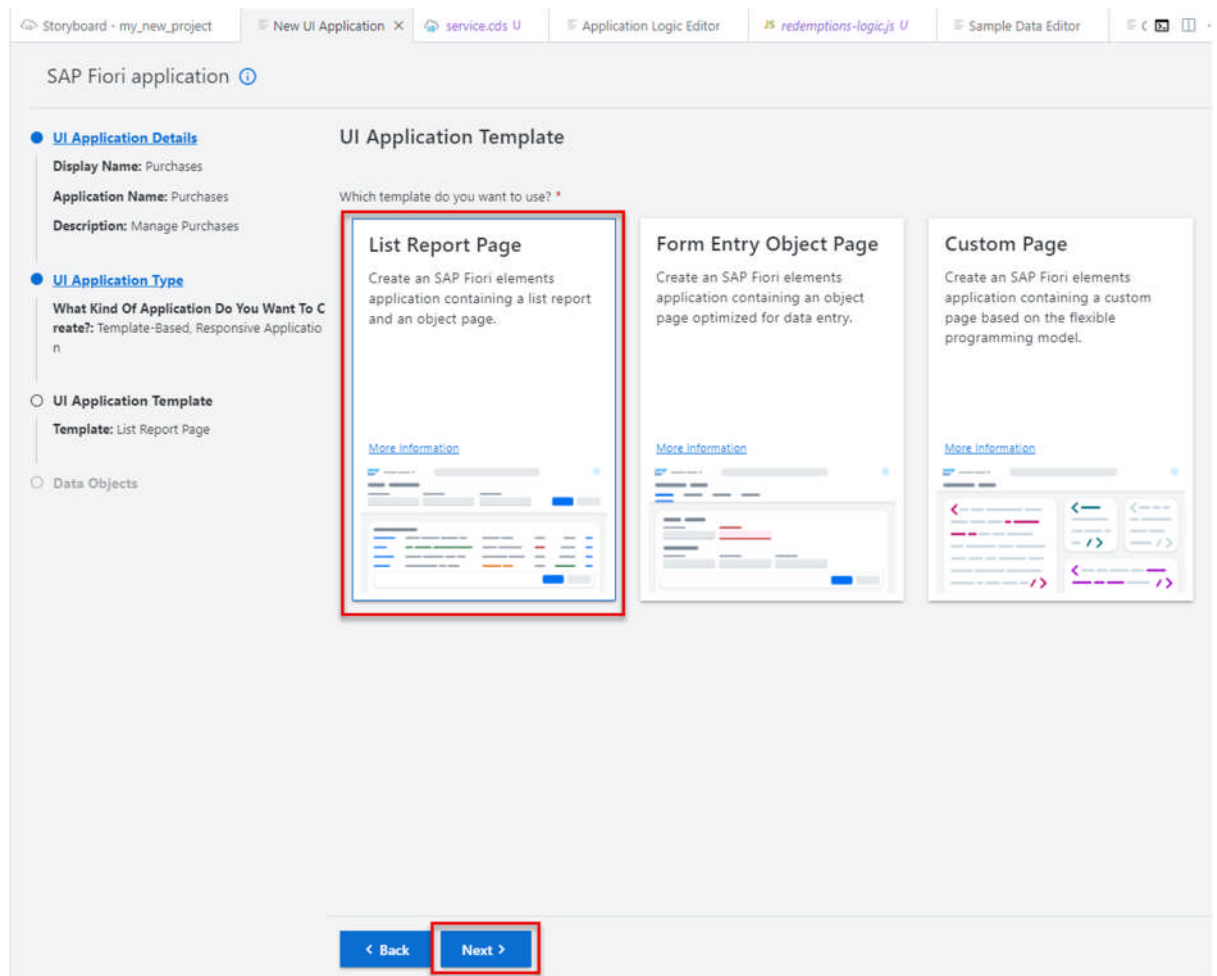
[More information](#)



< Back

Next >

4. Select **List Report Page** as the UI application template, and click **Next**.



5. Select **Purchases** as the **Main entity**, and click **Finish**. The page will be created now.

SAP Fiori application ⓘ

- **UI Application Details**
 - Display Name: Purchases
 - Application Name: Purchases
 - Description: Manage Purchases
- **UI Application Type**
 - What Kind Of Application Do You Want To Create?: Template-Based, Responsive Application
 - ☒ Yes ☐ No
 - ⓘ Basic value helps will also be created.
- **UI Application Template**
 - Template: List Report Page
- **Data Objects**
 - Main Entity: Purchases
 - Generate Annotations: Yes

Data Objects

Main entity *

Purchases ▼

Automatically add table columns to the list page and a section to the object page if none already exists?

☒ Yes ☐ No

ⓘ Basic value helps will also be created.

< Back Finish

It might take a few moments for the UI to be created because the dependencies need to be installed.

6. Repeat steps 2 through 5 to create additional UI apps for the **Customers** and the **Redemptions** entities.

Customer:

- o Display name: **Customers**
- o Description: **Manage Customers**

- o UI Application type: **Template-Based Responsive Application**
- o UI Application Template: **List Report Page**
- o Main Entity: **Customers**

Redemptions:

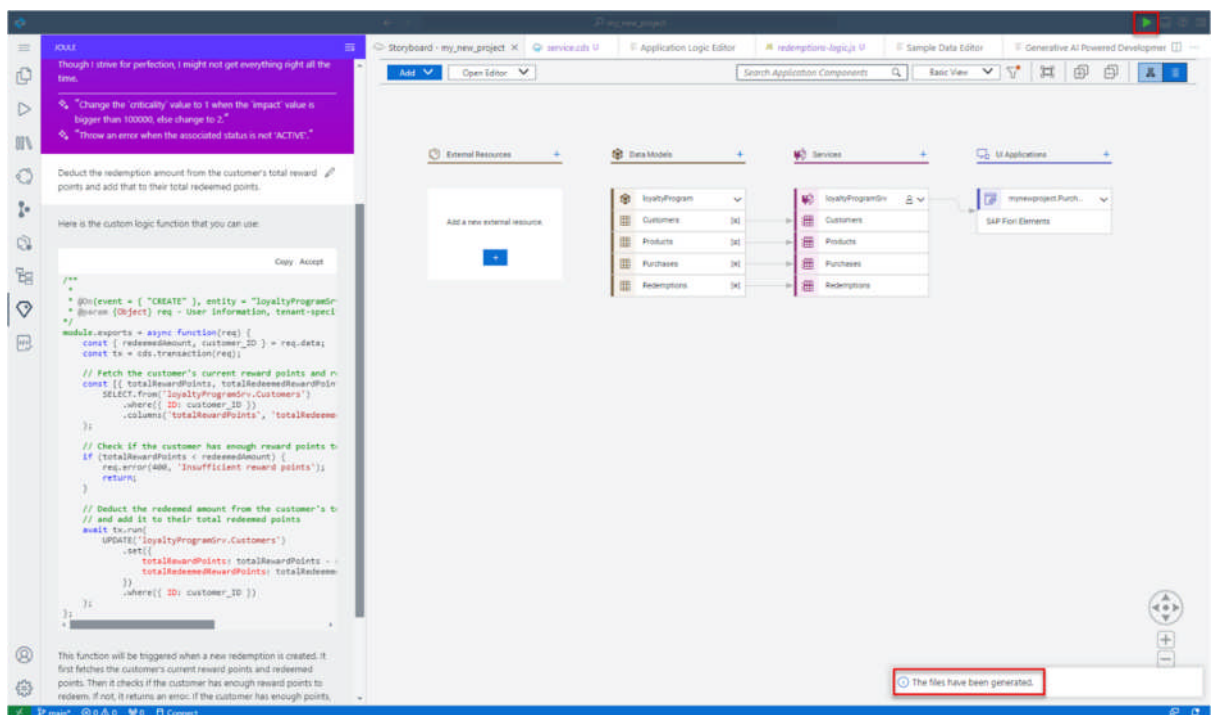
- o Display name: **Redemptions**
- o Description: **Manage Redemptions**
- o UI Application type: **Template-Based Responsive Application**
- o UI Application Template: **List Report Page**
- o Main Entity: **Redemptions**

And that's it! You've created an application.

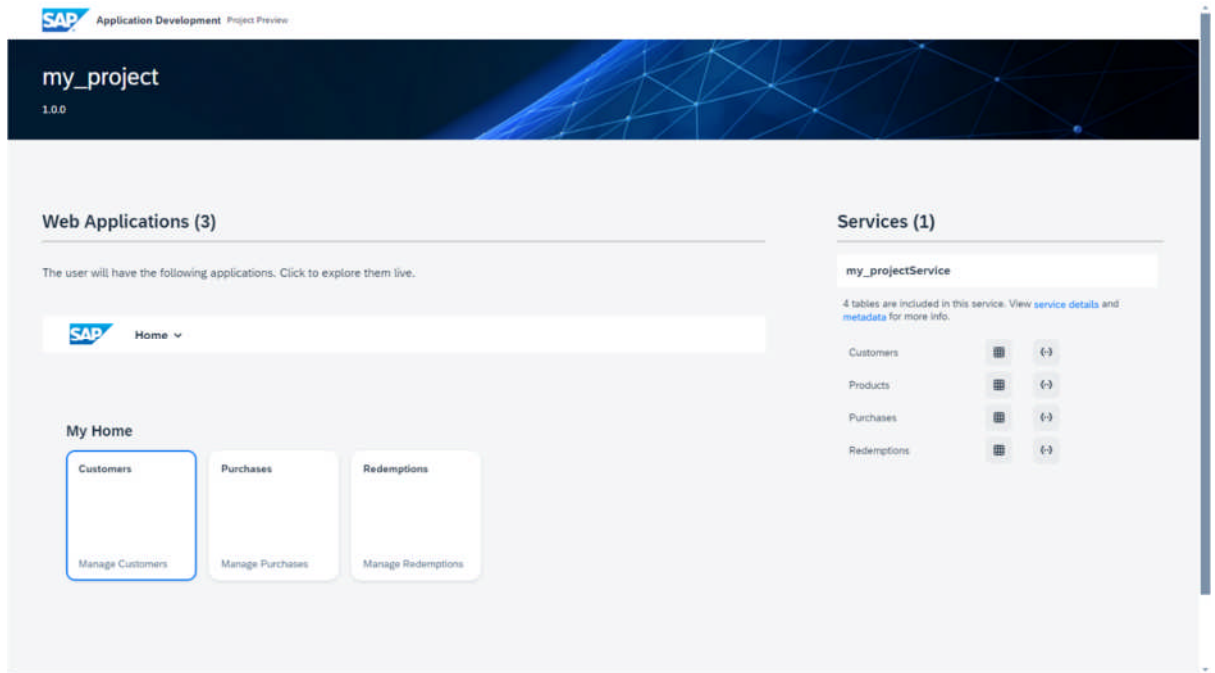
- To preview your application, once the files have been generated, go to the upper-right corner, and click



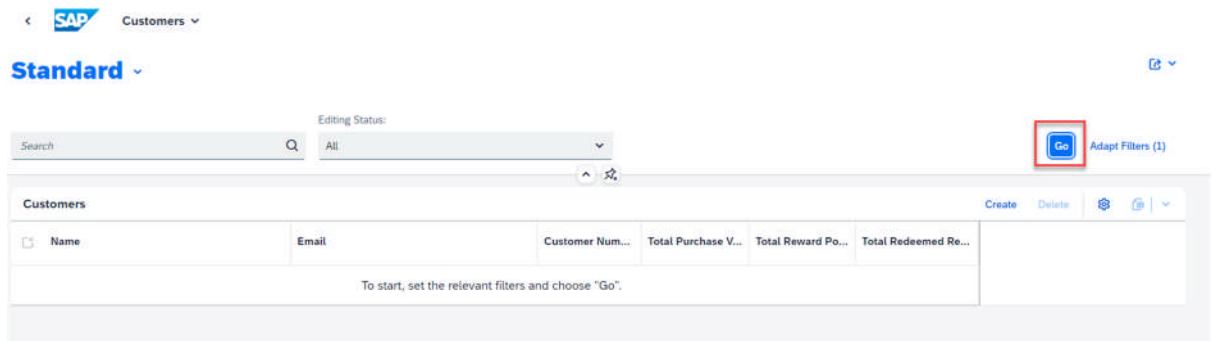
(Run and Debug).



The application's preview is displayed.



8. Click **Go**.



The customer information is displayed.

[SAP](#) Customers

Standard

Search Editing Status: All Adapt Filters (1)

Name	Email	Customer Num...	Total Purchase V...	Total Reward Po...	Total Redeemed Re...
☐ Emma Johnson	emmajohnson@example.com	2,345,678	6,000	400	200
☐ Sophia Williams	sophiawilliams@example.com	4,567,890	10,000	800	200
☐ John Doe	john doe@example.com	1,234,567	5,000	400	100
☐ Jane Smith	janesmith@example.com	7,654,321	8,000	600	200
☐ Michael Brown	michaelbrown@example.com	1,200,547	7,000	500	200
☐ Lucas Davis	lucasdavis@example.com	5,678,901	9,500	750	200
☐ Oliver Taylor	olivertaylor@example.com	3,456,789	9,000	700	200

9. From the dropdown list at the top of the page, select **Home** to go back and preview the other applications.

[SAP](#) Customers

Customers
 Home
 All My Apps

Search Editing Status: All Adapt Filters (1)

Name	Email	Customer Num...	Total Purchase V...	Total Reward Po...	Total Redeemed Re...
☐ Emma Johnson	emmajohnson@example.com	2,345,678	6,000	400	200
☐ Sophia Williams	sophiawilliams@example.com	4,567,890	10,000	800	200
☐ John Doe	john doe@example.com	1,234,567	5,000	400	100
☐ Jane Smith	janesmith@example.com	7,654,321	8,000	600	200
☐ Michael Brown	michaelbrown@example.com	1,200,547	7,000	500	200
☐ Lucas Davis	lucasdavis@example.com	5,678,901	9,500	750	200
☐ Oliver Taylor	olivertaylor@example.com	3,456,789	9,000	700	200